You can't define a new keyword, but you can fake it with a *Kernel Method (32)*. Kernel#using takes the managed resource as an argument. It also takes a block, which it executes. Regardless of whether the block completes normally, the ensure clause calls dispose on the resource to release it cleanly. There is no rescue clause, so any exception is still propagated to the code that calls Kernel#using.

Now that you've reviewed block basics, you can move to the second item on the list from *Today's Roadmap, on page 74*: closures.

Blocks Are Closures

Where you find there is more to blocks than meets the eye and you learn how to smuggle variables across scopes.

As Bill notes on a piece of scratch paper, a block is not just a floating piece of code. You can't run code in a vacuum. When code runs, it needs an *environment*: local variables, instance variables, self....

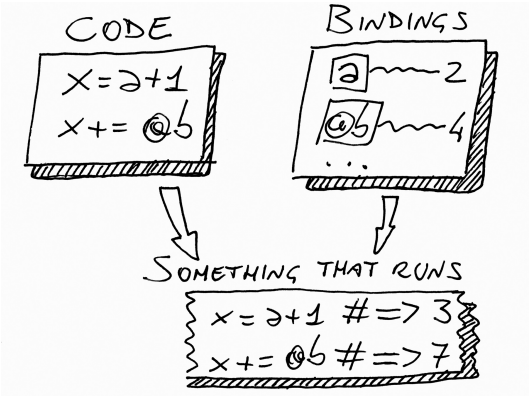


Figure 6—Code that runs is actually made up of two things: the code itself and a set of bindings.

Because these entities are basically names bound to objects, you can call them the *bindings* for short. The main point about blocks is that they are all inclusive and come ready to run. They contain both the code *and* a set of bindings.

You're probably wondering where the block picks up its bindings. When you define the block, it simply grabs the bindings that are there at that moment, and then it carries those bindings along when you pass the block into a method: